



MATH307

Fast Fourier Transform (FFT) Implementation and Its Applications

Contents

1	Abstract	2
2	Introduction	2
3	Problem Definition	3
4	Mathematical Methodology	3
4.1	Direct DFT as Matrix Multiplication	3
4.2	Radix-2 Cooley–Tukey FFT Derivation	3
4.3	Complexity Analysis	4
4.4	Filtering and Compression in the Frequency Domain	4
5	Implementation	4
5.1	Software Environment	4
5.2	Implemented Algorithms	5
5.3	Core FFT Code	5
5.4	Validation Strategy	5
6	Experimental Setup	6
6.1	Runtime and Accuracy Benchmark	6
6.2	Signal Filtering Experiment	6
6.3	Image Processing Experiment	6
7	Results	6
7.1	Runtime and Accuracy	6
7.2	Signal Frequency Analysis and Denoising	7
7.3	Image Filtering and Feature Emphasis	8
7.4	Frequency-Domain Compression	9
8	Discussion	10
8.1	Comparison of Techniques	10
8.2	Stability and Error Analysis	10
8.3	Parameter Effects	10
9	Conclusion	10
10	References	11

1 Abstract

The Discrete Fourier Transform (DFT) is a central numerical tool for converting discrete data from the time or spatial domain into the frequency domain. Its direct evaluation, however, requires $O(N^2)$ arithmetic operations for a one-dimensional signal of length N , which becomes expensive for large signals and images. This report studies the Fast Fourier Transform (FFT), especially the radix-2 Cooley–Tukey algorithm, as an efficient numerical method that reduces the computational complexity to $O(N \log N)$ while preserving the same mathematical transform.

The project combines mathematical derivation, algorithmic implementation, computational comparison, and applied experiments. A direct DFT, a recursive FFT, an inverse FFT, and two-dimensional FFT-based image processing routines were implemented in Python. The implementation was validated against NumPy’s FFT routines. Experiments measured runtime, relative numerical error, signal denoising accuracy, image denoising quality, and frequency-domain compression behavior. The results show that FFT produces relative errors near machine precision while scaling much more favorably than direct DFT. In the signal experiment, FFT-based low-pass filtering reduced mean squared error from 0.1579 to 0.0375, corresponding to a 6.24 dB improvement. In the image experiment, low-pass filtering improved PSNR from 20.65 dB to 21.99 dB while retaining only 8.43% of centered frequency coefficients. These findings demonstrate both the numerical efficiency and practical value of FFT for signal processing, image analysis, and AI-oriented preprocessing.

2 Introduction

Frequency-domain analysis is widely used in numerical analysis, engineering, signal processing, image processing, and artificial intelligence. Many physical and computational signals are easier to understand after decomposition into sinusoidal components. Periodic behavior, noise bands, edges, textures, and compression opportunities often become clearer in the frequency domain than in the original domain.

For a discrete signal $x = \{x_0, x_1, \dots, x_{N-1}\}$, the DFT is defined by

$$X_k = \sum_{n=0}^{N-1} x_n e^{-i2\pi kn/N}, \quad k = 0, 1, \dots, N-1. \quad (1)$$

The inverse transform reconstructs the original signal:

$$x_n = \frac{1}{N} \sum_{k=0}^{N-1} X_k e^{i2\pi kn/N}, \quad n = 0, 1, \dots, N-1. \quad (2)$$

Although Equation (1) is mathematically direct, evaluating all N frequency coefficients requires N^2 complex multiplications and additions. This cost limits direct DFT for large data. The FFT computes the same coefficients by exploiting algebraic structure in the complex roots of unity. The resulting $O(N \log N)$ complexity makes real-time audio processing, spectral analysis, medical imaging, scientific simulation, and image-based machine learning pipelines computationally practical.

This project addresses the MATH307 requirement to apply numerical analysis techniques to a computational science problem, solve or simulate the problem using software, compare methods, investigate parameter effects, analyze results, and demonstrate working output.

3 Problem Definition

The main computational problem is to evaluate the frequency representation of one-dimensional signals and two-dimensional images accurately and efficiently. For one-dimensional data, the task is to compute Equation (1). For image data $f(m, n)$ of size $M \times N$, the two-dimensional DFT is

$$F(u, v) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f(m, n) e^{-i2\pi(\frac{um}{M} + \frac{vn}{N})}, \quad (3)$$

with inverse

$$f(m, n) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) e^{i2\pi(\frac{um}{M} + \frac{vn}{N})}. \quad (4)$$

The project studies three connected questions:

1. How much faster is FFT than direct DFT as N increases?
2. Does the FFT preserve numerical accuracy compared with direct DFT and established library implementations?
3. How effectively can FFT support practical signal and image processing operations such as denoising, edge enhancement, and compression?

4 Mathematical Methodology

4.1 Direct DFT as Matrix Multiplication

Equation (1) can be written in matrix form as

$$\mathbf{X} = W_N \mathbf{x}, \quad (5)$$

where

$$(W_N)_{k,n} = e^{-i2\pi kn/N}. \quad (6)$$

This form is useful for implementation and for understanding why direct DFT costs $O(N^2)$: the dense matrix W_N has N^2 entries and each frequency coefficient depends on every input sample.

4.2 Radix-2 Cooley–Tukey FFT Derivation

Assume N is even. Split the input sequence into even-indexed and odd-indexed samples:

$$x_m^{(e)} = x_{2m}, \quad x_m^{(o)} = x_{2m+1}, \quad m = 0, \dots, \frac{N}{2} - 1.$$

Substituting this split into Equation (1) gives

$$X_k = \sum_{m=0}^{N/2-1} x_{2m} e^{-i2\pi k(2m)/N} + \sum_{m=0}^{N/2-1} x_{2m+1} e^{-i2\pi k(2m+1)/N} \quad (7)$$

$$= \sum_{m=0}^{N/2-1} x_m^{(e)} e^{-i2\pi km/(N/2)} + e^{-i2\pi k/N} \sum_{m=0}^{N/2-1} x_m^{(o)} e^{-i2\pi km/(N/2)} \quad (8)$$

$$= E_k + \omega_N^k O_k, \quad (9)$$

where $\omega_N = e^{-i2\pi/N}$, and E_k, O_k are DFTs of the even and odd subsequences. Because $E_{k+N/2} = E_k$ and $O_{k+N/2} = O_k$, the second half of the output is obtained by

$$X_{k+N/2} = E_k - \omega_N^k O_k, \quad k = 0, \dots, \frac{N}{2} - 1. \quad (10)$$

Recursively applying this decomposition gives the radix-2 FFT.

4.3 Complexity Analysis

Let $T(N)$ denote the number of operations for radix-2 FFT. Each recursion level computes two transforms of size $N/2$ and performs $O(N)$ combination work:

$$T(N) = 2T(N/2) + O(N). \quad (11)$$

By the Master Theorem, $T(N) = O(N \log_2 N)$. Direct DFT requires $O(N^2)$. The improvement factor is approximately

$$\frac{N^2}{N \log_2 N} = \frac{N}{\log_2 N},$$

which grows quickly as N increases.

4.4 Filtering and Compression in the Frequency Domain

Frequency-domain filtering multiplies the transform by a mask:

$$\hat{x}_{filtered} = \mathcal{F}^{-1}(M(k)\mathcal{F}(x)). \quad (12)$$

A low-pass mask keeps frequencies below a cutoff and suppresses high-frequency noise. A high-pass mask suppresses low frequencies and emphasizes rapid changes, which often correspond to edges in images.

For compression, many image frequency coefficients have small magnitude. Let F be a 2D FFT and let S be a subset of coefficients chosen to retain a target energy percentage:

$$\frac{\sum_{(u,v) \in S} |F(u,v)|^2}{\sum_{u,v} |F(u,v)|^2} \geq \eta. \quad (13)$$

The reconstructed image is obtained by inverse FFT after setting coefficients outside S to zero. This produces a tradeoff between storage and reconstruction quality.

5 Implementation

5.1 Software Environment

The experiments were implemented in Python using NumPy for numerical arrays and validation, Matplotlib for plots, and PIL for synthetic image generation. The complete code is stored in

`experiments/fft_experiments.py`.

The script writes generated figures, CSV files, and LaTeX table rows to `Report/generated/`.

5.2 Implemented Algorithms

The implementation contains:

- A direct DFT using the matrix definition.
- A recursive radix-2 FFT.
- An inverse FFT using the conjugate identity.
- Runtime and accuracy benchmarking against NumPy FFT.
- A signal denoising experiment using FFT low-pass filtering.
- A synthetic image experiment using 2D FFT for denoising, edge enhancement, and compression.

5.3 Core FFT Code

Listing 1: Recursive Cooley–Tukey FFT implementation used in the experiments.

```
def fft_recursive(x):
    x = np.asarray(x, dtype=np.complex128)
    n = x.size
    if n == 1:
        return x.copy()
    if n % 2 != 0:
        raise ValueError("fft_recursive requires a power-of-two length")
    even = fft_recursive(x[::2])
    odd = fft_recursive(x[1::2])
    factor = np.exp(-2j * np.pi * np.arange(n) / n)
    return np.concatenate([
        even + factor[: n // 2] * odd,
        even + factor[n // 2 :] * odd
    ])
```

5.4 Validation Strategy

The implementation was validated by comparing the direct DFT and recursive FFT against `numpy.fft.fft`. Relative error was measured using

$$\varepsilon_{rel} = \frac{\|X_{method} - X_{NumPy}\|_2}{\|X_{NumPy}\|_2}. \quad (14)$$

For denoising, mean squared error (MSE) was used:

$$\text{MSE} = \frac{1}{N} \sum_{n=0}^{N-1} (x_n - \hat{x}_n)^2. \quad (15)$$

For images, peak signal-to-noise ratio (PSNR) was also used:

$$\text{PSNR} = 20 \log_{10} \left(\frac{255}{\sqrt{\text{MSE}}} \right). \quad (16)$$

6 Experimental Setup

6.1 Runtime and Accuracy Benchmark

Complex random signals were generated for $N = 16, 32, \dots, 2048$. Direct DFT was benchmarked up to $N = 512$, since its $O(N^2)$ cost becomes unnecessarily slow for larger sizes. Recursive FFT and NumPy FFT were benchmarked for all sizes. The best time over repeated runs was recorded.

6.2 Signal Filtering Experiment

A clean signal was constructed from two meaningful frequency components:

$$x(t) = 1.15 \sin(2\pi 50t) + 0.65 \sin(2\pi 120t).$$

The noisy signal included a high-frequency interference component at 300 Hz and Gaussian noise. The FFT spectrum was filtered with a low-pass cutoff of 160 Hz, then reconstructed using inverse FFT. This tests whether FFT can identify and remove unwanted frequency content while preserving the target signal.

6.3 Image Processing Experiment

A 256×256 grayscale synthetic image was generated with smooth regions, edges, lines, and geometric structures. Gaussian noise was added to create a noisy image. The 2D FFT was used to:

- visualize the log-magnitude spectrum,
- apply low-pass denoising,
- apply high-pass edge emphasis,
- study compression by retaining increasing percentages of spectral energy.

7 Results

7.1 Runtime and Accuracy

Table 1: Runtime and relative error comparison for direct DFT, recursive FFT, and NumPy FFT. Times are best observed runtimes in milliseconds.

N	DFT time	Recursive FFT time	NumPy FFT time	DFT error	FFT error
16	0.023	0.127	0.0060	2.52×10^{-15}	2.95×10^{-16}
32	0.047	0.258	0.0060	6.90×10^{-15}	5.30×10^{-16}
64	0.374	0.530	0.0064	9.09×10^{-15}	4.90×10^{-16}
128	0.800	1.089	0.0074	2.11×10^{-14}	5.55×10^{-16}
256	3.138	2.227	0.0089	4.15×10^{-14}	6.56×10^{-16}
512	11.915	4.503	0.0201	8.63×10^{-14}	7.17×10^{-16}
1024	—	9.153	0.0176	—	7.95×10^{-16}
2048	—	18.586	0.0291	—	8.61×10^{-16}

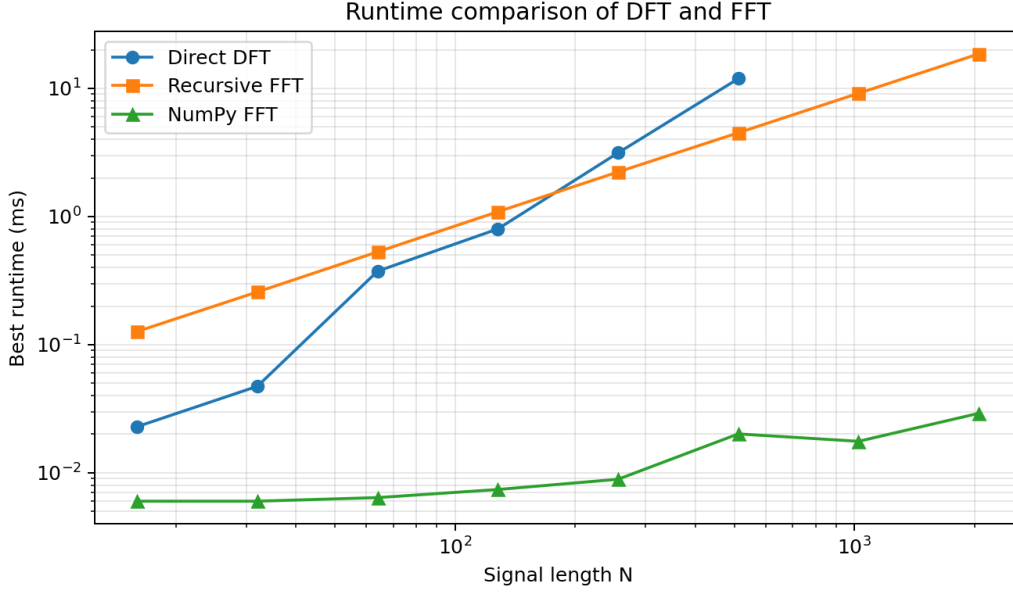


Figure 1: Runtime comparison on a logarithmic scale. The direct DFT curve grows much faster than both FFT implementations.

Table 1 and Figure 1 confirm the expected complexity behavior. Direct DFT is competitive only for very small N , where Python overhead and recursion overhead dominate. As N increases, the recursive FFT becomes faster than direct DFT. NumPy FFT is much faster than both because it uses optimized compiled routines and low-level memory-efficient implementations.

The relative errors are near 10^{-15} , which is consistent with double-precision floating-point arithmetic. This shows that the recursive FFT is not an approximation to DFT; it computes the same transform up to roundoff error. Small differences arise from the order of floating-point operations.

7.2 Signal Frequency Analysis and Denoising

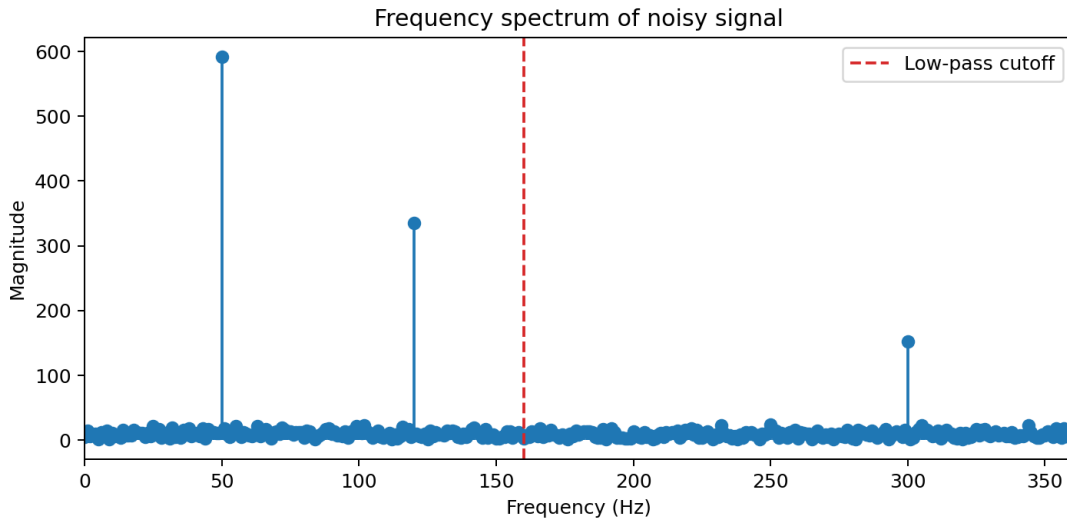


Figure 2: Frequency spectrum of the noisy signal. Dominant peaks occur at 50, 120, 300 Hz, matching the designed clean components and the injected interference.

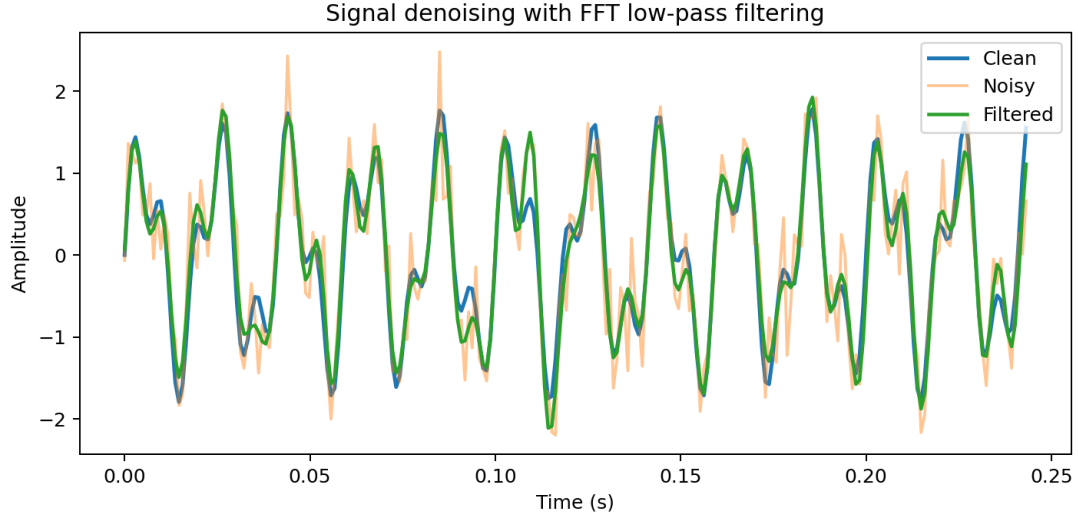


Figure 3: Time-domain comparison of the clean, noisy, and FFT-filtered signals. Low-pass filtering removes much of the high-frequency interference.

The FFT spectrum correctly identifies the main frequency content at 50, 120, 300 Hz. The 300 Hz component is above the chosen 160 Hz cutoff, so it is removed by the low-pass filter. The signal MSE decreased from 0.1579 to 0.0375, a 6.24 dB improvement. This demonstrates how FFT allows a numerical method to transform a difficult time-domain denoising problem into a simpler frequency-domain masking problem.

7.3 Image Filtering and Feature Emphasis

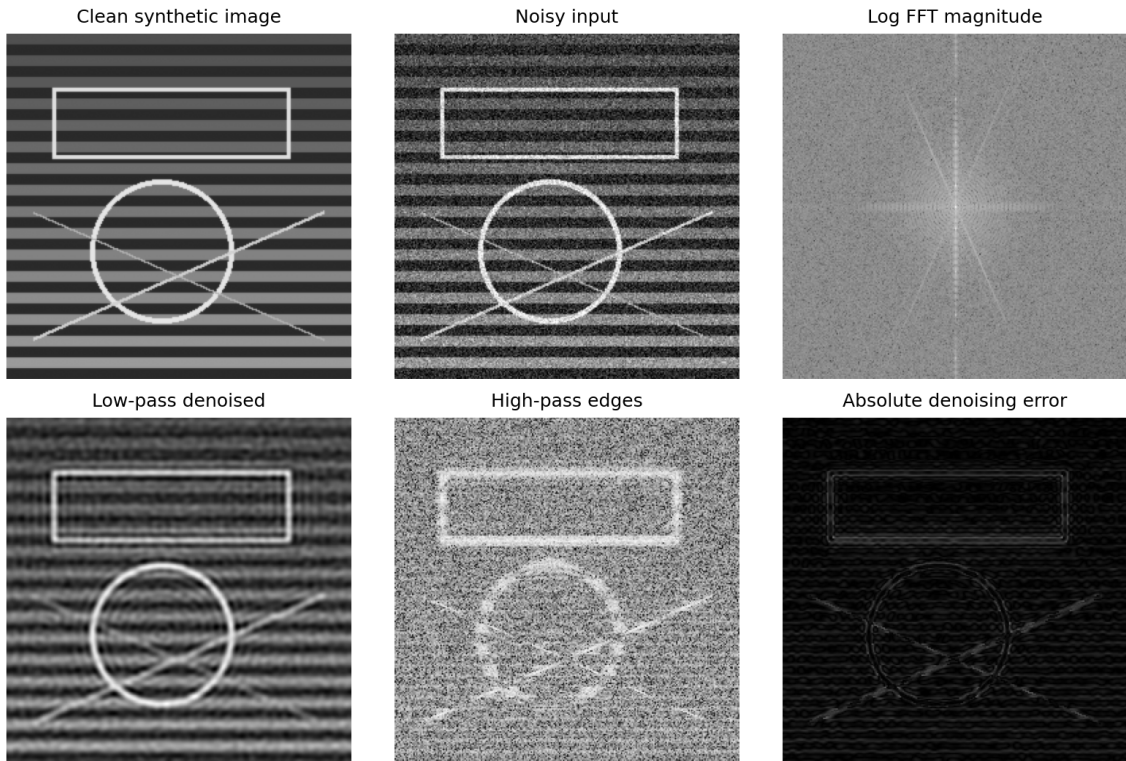


Figure 4: 2D FFT image processing results: clean synthetic image, noisy input, log-magnitude spectrum, low-pass denoising, high-pass edge emphasis, and absolute denoising error.

The 2D FFT experiment shows that low frequencies encode the broad structure of the image, while high frequencies encode sharp transitions, edges, and noise. The low-pass filter improved PSNR from 20.65 dB to 21.99 dB and reduced MSE from 560.29 to 411.54. The filter retained only 8.43% of the centered frequency coefficients, illustrating the concentration of useful image energy in low-frequency components.

The high-pass result emphasizes edges and fine structures. This is useful for feature extraction and preprocessing in image analysis, although high-pass filtering alone is not a denoising method because noise also has significant high-frequency content.

7.4 Frequency-Domain Compression

Table 2: Image compression tradeoff by retaining coefficients required to preserve target spectral energy.

Target energy	Coefficients retained	MSE	PSNR (dB)
50%	0.00%	2360.61	14.40
90%	0.04%	995.87	18.15
95%	0.58%	499.83	21.14
98%	2.99%	200.03	25.12
99%	6.13%	100.05	28.13

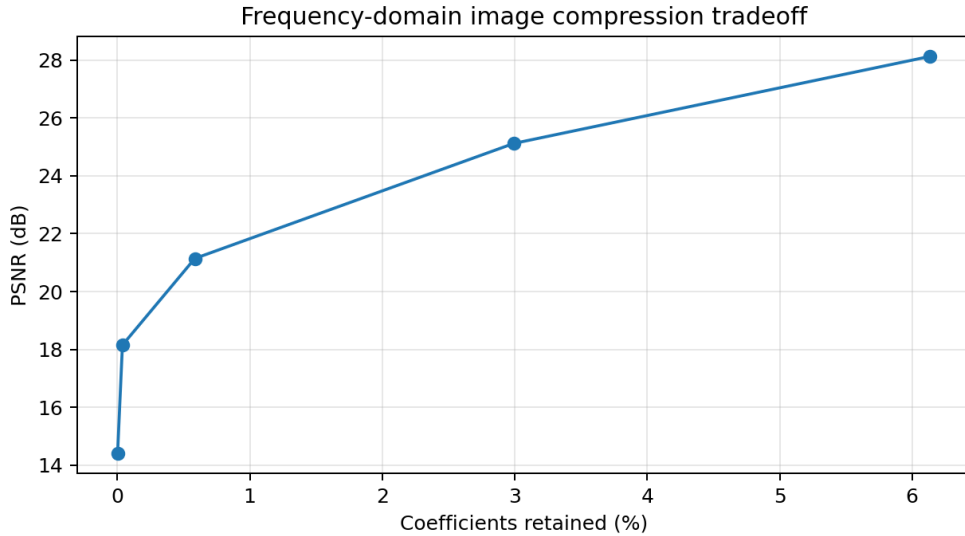


Figure 5: Compression quality improves as more frequency coefficients are retained.

Table 2 shows a clear tradeoff. Retaining more spectral energy improves PSNR, but it also requires more coefficients. The 98% energy case retained only 2.99% of coefficients while achieving 25.12 dB PSNR. The 99% energy case retained 6.13% of coefficients and increased PSNR to 28.13 dB. This supports the practical observation that many images can be represented compactly in the frequency domain, especially when they contain smooth regions and structured edges.

8 Discussion

8.1 Comparison of Techniques

Direct DFT is simple and mathematically transparent, but it scales poorly because every output frequency directly sums over every input sample. Recursive FFT is mathematically equivalent but reorganizes the computation so repeated substructure is reused. NumPy FFT is the most efficient implementation in the experiment because it uses optimized production algorithms, but the custom recursive FFT is valuable educationally because it exposes the numerical method.

For filtering, the time-domain signal is hard to clean by direct inspection because random noise and sinusoidal interference overlap in time. In the frequency domain, the unwanted 300 Hz component is clearly separated from the target 50 Hz and 120 Hz components. This explains the strong MSE reduction after masking.

For images, low-pass filtering improves noise robustness but can blur edges because edges require high-frequency coefficients. High-pass filtering highlights edges but also emphasizes high-frequency noise. Compression reveals a different tradeoff: retaining a small fraction of high-energy coefficients may preserve much of the image, but aggressive compression introduces reconstruction error.

8.2 Stability and Error Analysis

The FFT is backward stable for practical purposes: computed results are close to the exact DFT of a slightly perturbed input. The measured relative errors near 10^{-15} indicate that floating-point roundoff is small for the tested sizes. Error grows slowly with N because recursive combinations accumulate floating-point operations, but the growth remains acceptable in double precision.

Filtering error depends more on the modeling choice than on numerical roundoff. If the cutoff is too low, important signal components are removed. If it is too high, noise remains. Similarly, image low-pass filtering improves PSNR only when the removed high-frequency content is mostly noise; if the image contains essential fine texture, excessive smoothing would reduce quality.

8.3 Parameter Effects

The experiments investigated three parameter families:

- Signal size N : larger N increases the advantage of FFT over DFT.
- Frequency cutoff: the 160 Hz cutoff preserved the 50 Hz and 120 Hz components while suppressing 300 Hz interference.
- Compression energy threshold: higher retained energy improves PSNR but uses more coefficients.

These parameter effects are important because numerical methods are rarely used in isolation; their performance depends on problem scale and modeling choices.

9 Conclusion

This project implemented and analyzed the FFT as a numerical method for efficient frequency-domain computation. The mathematical derivation showed how radix-2 Cooley–Tukey decomposition reduces DFT complexity from $O(N^2)$ to $O(N \log N)$. The implementation validated

this improvement experimentally and showed near-machine-precision agreement with NumPy's FFT.

The application experiments demonstrated that FFT is not only faster than direct DFT but also practically useful. In one-dimensional signals, FFT-based filtering identified dominant frequencies and reduced denoising error substantially. In images, 2D FFT enabled spectral visualization, denoising, edge enhancement, and compression analysis. Overall, FFT is a strong example of how numerical analysis converts mathematical structure into computational efficiency and practical scientific insight.

10 References

References

- [1] J. W. Cooley and J. W. Tukey, "An Algorithm for the Machine Calculation of Complex Fourier Series," *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [2] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed., Pearson, 2010.
- [3] S. W. Smith, *The Scientist and Engineer's Guide to Digital Signal Processing*, California Technical Publishing, 1997.
- [4] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018.
- [5] NumPy Developers, "Discrete Fourier Transform (`numpy.fft`) Documentation."
- [6] R. N. Bracewell, *The Fourier Transform and Its Applications*, 3rd ed., McGraw-Hill, 2000.